Ders 5 Genişletilmiş Notu: Öğrenci Sunumları: CQRS Görünüm Tasarımı, Uçtan Uca Testler, Güvenli Servisler ve Microservice Chassis

Ders 5 Genişletilmiş Notu: Öğrenci Sunumları: CQRS Görünüm Tasarımı, Uçtan Uca Testler, Güvenli Servisler ve Microservice Chassis

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. CQRS view modülü ve tasarım gereksinimleri
- 2. View veritabanının seçimi ve güncelleme işlemleri
- 3. Veri erişim modülü: eşzamanlılık, idempotentlik ve tutarlılık
- 4. CQRS view ekleme ve güncelleme
- 5. Uçtan uca (end-to-end) testler: tanım, yer ve zorluklar
- 6. End-to-end testlerin tasarımı: user journey ve Gherkin
- 7. End-to-end testlerin çalıştırılması ve performansı
- 8. Üretime hazır servisler ve güvenliğin dört yönü
- 9. Monolitik uygulamada güvenlik ve mikroservislere uygulanamaması
- 10. API Gateway'de kimlik doğrulama ve yetkilendirme
- 11. Belirteç türleri: opak belirteçler ve JWT
- 12. Mikroservis mimarisinde OAuth 2.0
- 13. Microservice Chassis deseni
- 14. Service Mesh

Bu ders dört öğrenci sunumundan oluşur. İlk sunum, önceki derste yapılamayan CQRS görünümlerinin (view) tasarımıyla ilgili sunumdur (Bölüm 1-4). Ardından o günün planlanan sunumlarına geçilir: uçtan uca testler (Bölüm 5-7), güvenli servisler geliştirmek (Bölüm 8-12) ve Microservice Chassis ile Service Mesh (Bölüm 13-14). Planlanan iki sunum sonraki haftaya kalır. Sunan kişiler bu notta "bir öğrenci" diye anılır. Hoca bu derste sunumları yönetmiş; CQRS sunumunun ardından CQRS'e önceki hafta da bakıldığını, bu sunumla konunun bir kez daha hatırlandığını söylemek dışında akademik bir ek açıklama yapmamıştır.

> **Ek bilgi (derste anlatılmadı):** Sunumlardaki şekiller, tablolar ve listeler (FTGO, Order History View, Listing 10.17 vb.) Chris Richardson'ın *Microservices Patterns* kitabından alınmıştır. Bu kitapta CQRS görünümlerinin tasarımı büyük olasılıkla yedinci bölümde, uçtan uca testler onuncu bölümde, güvenlik ile Microservice Chassis ve Service Mesh ise on birinci bölümde ("Developing production-ready services") yer alır.

1. CQRS view modülü ve tasarım gereksinimleri

Öğrenci sunumu dört bölüme ayırır (ekrandan, 00:01:40): giriş (CQRS view modülü, modül tasarımı gereksinimleri), veritabanı (veritabanı seçimi, SQL ve NoSQL, güncelleme işlemleri), veri erişim modülü tasarımı (veri erişim modülü, eşzamanlı güncellemeler, idempotent güncellemeler, tutarlı veri sağlama) ve CQRS view ekleme ve güncelleme (arşivlenmiş event'ler

ile oluşturma, artımlı olarak oluşturma).

1.1 CQRS view modülünün yapısı

Önceki derste CQRS'in komut ve sorgu taraflarını ayırdığı, sorgu tarafının komut tarafının yayınladığı event'lere abone olarak kendi görünümünü güncellediği anlatılmıştı. Bu sunum o sorgu tarafının, yani bir CQRS view modülünün içini açar. Modülde iki aktör vardır:

- Query API: dışarıdan gelen sorguları uygular.
- Event handler'lar: diğer servislerin event'lerine abone olur ve görünümü günceller.

İkisi de güncelleme ve sorgu işlemlerini bir veri erişim modülü (Data Access) üzerinden view veritabanında yapar. Öğrencinin vurgusuyla veri erişim modülü burada aracıdır; Query API ve event handler'lar view veritabanına doğrudan erişemez.

1.2 Modül tasarımı gereksinimleri

Slayttaki dört gereksinim şunlardır (ekrandan, 00:02:20):

- Veritabanı: kullanılacak veritabanı ve şeması belirlenmelidir.
- Data Access: güncellemeler idempotent olmalı ve eşzamanlı güncellemeler sistemi bozmamalıdır.
- Entegrasyon: var olan bir sisteme yeni bir view modülü eklenecekse ya da şema değişecekse, view veritabanını oluşturacak bir mekanizma bulunmalıdır.
- Gecikme: istemciler replikasyon gecikmesinden etkilenmemelidir.

Sunumun geri kalanı bu dört gereksinimi sırayla ayrıntılandırır.

2. View veritabanının seçimi ve güncelleme işlemleri

2.1 Veritabanı seçimi

Öğrenciye göre bir view veritabanında en önemli şey, sorgulama ve güncelleme işlemlerinin verimli yapılmasıdır. Bu yüzden sorgu ve güncellemelerin karakteristiği veritabanı seçiminde büyük rol oynar. Slayttaki tablo, ihtiyaca göre hangi veritabanının kullanılacağını örneklerle verir (ekrandan, 00:03:15):

İhtiyaç	Kullanılacak	Örnek
Birincil anahtarla JSON nesnesi arama	MongoDB, DynamoDB gibi doküman deposu ya da Redis gibi anahtar-değer deposu	Müşteri başına bir MongoDB dokümanı tutarak sipariş geçmişi
Sorguyla JSON nesnesi arama	MongoDB ya da DynamoDB gibi doküman deposu	Müşteri görünümünü MongoDB ya da DynamoDB ile gerçekleştirme
Metin sorguları	Elasticsearch gibi metin arama motoru	Sipariş başına bir Elasticsearch dokümanıyla siparişlerde metin araması
Graf sorguları	Neo4j gibi graf veritabanı	Müşteri, sipariş ve diğer verilerden graf tutarak dolandırıcılık tespiti
Geleneksel SQL raporlama/BI	İlişkisel veritabanı (RDBMS)	Standart iş raporları ve analizler

Öğrenci tablodan iki örnek okur: birincil anahtarla JSON nesnesi aranacaksa MongoDB ya da DynamoDB, metin sorgusu yapılacaksa Elasticsearch gibi bir arama motoru.

2.2 SQL ve NoSQL

Slayttaki karşılaştırma şöyledir (ekrandan, 00:04:00). NoSQL veritabanları, SQL'in ölçeklenebilirlikteki kısıtları nedeniyle ortaya çıkmıştır; kullanım alanına göre daha esnek veri modeli,

daha iyi performans ve ölçeklenebilirlik sunarlar, ancak SQL kadar karmaşık işlemleri ve sorguları yapamazlar.

CQRS modüllerinde yapılan işlemler genelde basit işlemlerdir. Bu yüzden NoSQL'in karmaşık işlemlerdeki dezavantajı burada hissedilmez, esnek veri modeli ve performans avantajı kullanılır; NoSQL veritabanları CQRS modülleri için iyi bir seçimdir.

Buna rağmen SQL veritabanları da hâlâ tercih edilebilir. Öğrencinin saydığı nedenler:

- Modern donanımlarda SQL veritabanlarının performansı iyi olabilir.
- Geliştiricilerin ve BT ekiplerinin halihazırda SQL tecrübesi vardır.
- Raporlama araçları SQL'i destekler.
- SQL veritabanları bazı ilişkisel olmayan özellikleri de içerebilir.

2.3 Güncelleme işlemleri

Güncellemeler genellikle birincil anahtar (primary key) kullanılarak yapılır (ekrandan, 00:05:20). Örneğin siparişler tablosunda bir sipariş güncellenecekse, sipariş kimliği anahtarıyla ilgili satır bulunur ve güncellenir.

Bazı event'ler ise yabancı anahtar (foreign key) üzerinden güncelleme gerektirir. Öğrencinin örneğinde TeslimatOluşturuldu event'i sipariş kimliğini içerebilir, bu durumda sipariş doğrudan güncellenir. Ancak TeslimatAlındı event'i sipariş kimliğini içermeyebilir; o zaman sipariş, teslimat kimliği kullanılarak, yani yabancı anahtar üzerinden güncellenmelidir.

Slayta göre ilişkisel veritabanları ve MongoDB yabancı anahtarla güncellemeyi indeksleme sayesinde verimli yapabilir. Diğer NoSQL veritabanlarının çoğunda ise bu daha karmaşık işlemler gerektirebilir.

3. Veri erişim modülü: eşzamanlılık, idempotentlik ve tutarlılık

3.1 Veri erişim modülünün görevi

Event handler'lar ve Query API veritabanında işlem yapmak için veri erişim modülünü kullanır. Modülün içinde veri erişim nesnesi (DAO, Data Access Object) ve yardımcı sınıflar bulunur. Event handler'ların güncelleme işlemleri ve Query API'nin sorgu işlemleri DAO'nun görevidir. DAO ayrıca veritabanı API'si ile yüksek seviyeli koddaki veri tipleri arasında köprü görevi görür. Giriş bölümündeki gereksinimlere göre bu modül eşzamanlı güncellemeleri idare edebilmeli ve güncellemelerin idempotent olmasını sağlamalıdır.

3.2 Eşzamanlı güncellemeler

DAO, aynı veritabanı kaydına yapılabilecek eşzamanlı güncellemeleri sorunsuz halletmelidir (ekrandan, 00:06:40). Öğrencinin örneğinde bir sipariş event'i ile bir teslimat event'i aynı anda gelir ve ikisi de aynı siparişi günceller. İki güncelleme aynı anda yapılmaya çalışılırsa biri diğerinin üstüne yazabilir ve veri kaybolur. Bunu önlemek için iyimser (optimistic) ya da kötümser (pessimistic) kilitleme kullanılır; böylece iki işlem aynı kaydı aynı anda güncelleyemez ve veri kaybolmaz.

> **Ek bilgi (derste anlatılmadı):** Kötümser kilitlemede kayıt okunurken kilitlenir ve işlem bitene kadar başka bir işlem onu değiştiremez. İyimser kilitlemede kilit tutulmaz; kayıtta bir sürüm numarası bulunur ve güncelleme yalnız okunan sürüm hâlâ geçerliyse yapılır, değilse işlem yeniden denenir. Çakışmanın seyrek olduğu durumlarda iyimser kilitleme genellikle daha verimlidir.

3.3 İdempotent güncellemeler

Bir event handler aynı event ile birden çok kez tetiklenebilir (ekrandan, 00:08:30). Event handler'ın idempotent olması için aynı event birden çok kez tetiklendiğinde de doğru sonuca ulaşılabilmesi gerekir; öğrencinin ifadesiyle event bir kez de beş kez de tetiklense sonuç doğruysa handler idempotenttir.

Slayttaki örnek (Order History View) zaman ekseninde dört event gösterir; ilk iki paket bir ağ hatası yüzünden tekrarlanmıştır:

```
Zaman ------------------------------------------------------------->
Delivery picked up   Delivery delivered   Delivery picked up   Delivery delivered
OrderId: 123         OrderId: 123         OrderId: 123         OrderId: 123
State: PICKED_UP     State: DELIVERED     State: PICKED_UP     State: DELIVERED
                                          ^ Temporarily out of date
```

Görünüm önce PICKED_UP, sonra DELIVERED durumuna geçer. Tekrarlanan "picked up" event'i yüzünden yeniden PICKED_UP olur ve son "delivered" event'i gelene kadar bu durumda kalır. Bu süre boyunca görünüm geçici olarak güncelliğini yitirmiştir. Slayta göre idempotentlik sağlanmazsa aynı event'lerin tekrarı durumunda veriler geçici olarak güncelliğini kaybedebilir.

Bu durum sistem için kabul edilemez ya da kritikse, event handler tekrarlanan event'leri tespit edecek bir mekanizmaya sahip olmalıdır. Öğrencinin anlattığı mekanizma:

- Her event için eşsiz bir kimlik (ID) oluşturulur.
- Event handler işlediği kimlikleri kaydeder.
- Yeni bir event geldiğinde kimliği daha önce gelenler arasında var mı diye bakılır. Yoksa event işlenir, varsa işlenmez.
- Bütün kimlikleri saklamak gerekmez. Kimlikler sıralı artıyorsa gelen en büyük kimliği saklamak yeterlidir; ondan küçük kimlikli event'ler zaten daha önce gelmiştir.

3.4 Tutarlı veri sağlama

Son gereksinim, istemcinin replikasyon gecikmesinden etkilenmemesidir (ekrandan, 00:10:00). İstemci bir güncelleme komutu gönderip hemen ardından sorgu yaparsa ve view veritabanı güncellenmeden sorgu işlenirse, istemciye güncellenmemiş veri döner. Bu, istemci ile sistem arasında bir tutarsızlıktır.

Verinin tutarlı olduğunu garantilemek için:

- Güncelleme komutunun cevabında istemciye bir token verilir. Token, güncellemeyi yapan event'in kimliğini içerir.
- İstemci sorgu yaparken bu token'ı da gönderir.
- Sistem bu kimlikle güncellemenin yapılıp yapılmadığına bakar. Yapılmadıysa hata döndürür, yapıldıysa güncel veriyi döndürür.

4. CQRS view ekleme ve güncelleme

Bir uygulamanın ömrü boyunca yeni bir view eklemek ya da var olanı güncellemek mutlaka gerekir: yeni bir sorgu desteği eklenebilir, bir hata çıkıp düzeltilirken görünüm değişebilir ya da veritabanı şemasına yeni bir şey gelebilir.

Yeni bir view eklemek için girişteki diyagramdaki parçaları gerçekleştirmek yeterlidir: sorgu modülü ve veritabanı hazırlanır, event handler'lar ilgili event'lere abone edilir ve görünüm zamanla güncel hale gelir. Var olan bir view'ı baştan oluşturmak ya da güncellemek için ise iki yol vardır.

4.1 Arşivlenmiş event'ler ile oluşturma

Bu yöntemde bütün event'ler, yeni oluşturulan bir view'ı güncel hale getirmek için arşivlenir (ekrandan, 00:11:40). Sorun, mesaj kuyruklarının event'leri kalıcı tutmamasıdır:

- RabbitMQ gibi geleneksel mesaj kuyrukları mesajı işlendikten sonra siler.
- Daha modern olan Apache Kafka'da ayarlanabilir bir mesaj tutma süresi vardır, ama mesajları sonsuza kadar tutamaz.

Bu yüzden arşivleme için slayta göre AWS S3 ya da daha ölçeklenebilir olan Apache Spark gibi servisler kullanılmalıdır.

4.2 Artımlı olarak oluşturma

Arşiv yönteminde gelen her event arşivlendiği için, zaman geçtikçe yeni bir view oluşturmak için gereken zaman ve kaynak artar. Bunun yerine iki adımlı artımlı (incremental) algoritma kullanılır:

1. Periyodik olarak her aggregate için, bir önceki snapshot'tan sonra gelen event'lerden yeni bir snapshot oluşturulur. Yani $t-1$ anındaki snapshot ve sonrasındaki event'lerden t anındaki snapshot elde edilir.
2. View, bu snapshot'lar ve snapshot'lardan sonra gelen event'ler kullanılarak oluşturulur.

Öğrenciye göre bu yöntem checkpoint almaya benzer; bu yüzden önceki yöntemdeki gibi zaman ve kaynak ihtiyacı sürekli artmaz.

5. Uçtan uca (end-to-end) testler: tanım, yer ve zorluklar

İkinci sunum, önceki derste başlayan mikroservis test sunumlarının devamıdır ve end-to-end testlerin ne olduğunu, zorluklarını, tasarımını ve performansını anlatır. Giriş bölümünün başlıkları: end-to-end test nedir, end-to-end testlerin yeri ve önemi, amaç ve kapsam (ekrandan, 00:15:00).

5.1 Tanım

End-to-end testler, bir uygulamanın bütün bileşenlerinin birlikte çalıştığını doğrulayan testlerdir. Uygulamanın gerçek kullanım senaryolarını simüle eder, kullanıcı etkileşimlerinin ve iş süreçlerinin doğru çalıştığını test eder. Amaçları, sistemin uçtan uca entegrasyonunun ve işlevselliğinin beklendiği gibi çalıştığını garanti altına almaktır.

5.2 Test piramidindeki yeri

End-to-end testler test piramidinin en üst katmanında yer alır ve en karmaşık testlerdir. Piramidin katmanları:

- En alt: birim (unit) testleri. Küçük ve hızlı testlerdir, temel bileşenleri test eder.
- Orta: entegrasyon testleri. Sistemin bileşenlerinin, yani hizmetlerin birbiriyle doğru çalışıp çalışmadığını test eder.
- En üst: end-to-end testler. Uygulamanın tamamını, sistemin bir bütün olarak doğru çalışıp çalışmadığını test eder.

End-to-end testler bütün bileşenleri kapsadığı için yavaş, kırılgan ve geliştirmesi zaman alıcıdır; bu yüzden en üst katmandadır ve az sayıda yazılmaları tavsiye edilir.

5.3 Rol, kapsam ve öneri

Rolü, uygulamanın işlevselliğini bir bütün olarak ve kullanıcı perspektifinden doğrulamaktır. Kapsamı bütün sistemin çalışabilirliğidir; altyapı hizmetleri dahil birden fazla hizmetin entegrasyonunu kapsar. Öneri, bu testlerin sayısının mümkün olduğunca az tutulmasıdır. Bunların yerine user journey testleri gibi daha kapsamlı ve verimli stratejiler benimsenmelidir.

5.4 Zorluklar

Öğrenci zorlukları dört grupta toplar:

- Yavaşlık: testler bütün uygulamayı ve altyapıyı çalıştırır. Birçok hizmetin başlatılması ve senaryoların tamamlanması zaman alır, bu da geliştirme döngüsünü uzatır.
- Kırılganlık: çok sayıda hareketli parça (hizmetler, altyapı) vardır. Bir ya da daha fazla hizmet çalışmazsa test başarısız olur; testlerin güvenilirliği azalır ve sık sık yanlış pozitif sonuçlar üretilebilir.
- Geliştirme ve bakım güçlüğü: testlerin yazılması ve sürdürülmesi zaman alır. Yapılan bir değişiklik birçok testi etkileyip güncellemeyi zorunlu kılabilir; bakım maliyeti artar.
- Ölçekleme sorunları: yüzlerce hizmet ya da test içeren sistemlerde testlerin yürütülmesi çok uzun sürebilir.

Bu zorluklara karşı test senaryoları optimize edilir (örneğin user journey testleri), daha az ama daha etkili end-to-end test yazılır ve test altyapısı Gradle Docker Compose gibi araçlarla otomatikleştirilir.

6. End-to-end testlerin tasarımı: user journey ve Gherkin

6.1 User journey testleri

Slayttaki tasarım başlıkları şunlardır (ekrandan, 00:20:00):

- User journey testi: kullanıcıların taklit edildiği senaryolar (örneğin sipariş oluşturma). Avantajları: az ama kapsamlı test, test sürecinin kısalması.
- Test senaryolarını belirleme: gerçek kullanıcı davranışlarına dayanarak kritik yolculukların belirlenmesi, işlevlerin sistem üzerindeki etkisinin analiz edilmesi.
- Görsel destek ve örnek: Gherkin formatında senaryo örneği. Kullanıcı geçerli bir kredi kartıyla sipariş verir, siparişi revize ederek ek öğeler ekler, son olarak siparişi iptal eder.
- Öneri ve dikkat edilmesi gerekenler: kritik iş paketleri için birkaç iyi senaryo, aşırı karmaşık testlerden kaçış, testlerin birbirini etkilemesinden kaçınma.

Bir user journey testi, bir kullanıcının sistemdeki yolculuğuna karşılık gelir. Öğrencinin örneğinde sipariş oluşturma, revize etme ve iptal etme ayrı ayrı test edilmek yerine üçünü birden yapan tek bir test yazılır. Bu yaklaşım yazılması gereken test sayısını azaltır ve test yürütme süresini kısaltır.

6.2 Gherkin

Gherkin, iş dünyasındaki paydaşların anlayabileceği bir dille test senaryoları yazmayı sağlayan alana özgü bir dildir (DSL). Testleri Given/When/Then yapısıyla düzenler. End-to-end testler, kabul testleri gibi iş dünyasına yönelik testlerdir; bu yüzden onları iş dünyasındaki kişilerin anlayabileceği üst düzey bir DSL ile yazmak mantıklıdır.

Öğrencinin benzetmesiyle, biri A dilini, karşısındaki C dilini bilse de Gherkin ile yazılmış senaryoyu okuyarak ikisi de testin ne yaptığını anlar; ortak bir şeyi anlamak için aynı dil konuşulur. Böylece teknik olmayan ekipler de test senaryolarını anlayabilir. Slayttaki avantajlar: iş birliği kolaylığı, doğrudan anlaşılabilirlik ve otomasyon.

Slayttaki örnek, kitaptaki “Listing 10.17 A Gherkin-based specification of a user journey”dir (ekrandan, 00:21:00):

```
Feature: Place Revise and Cancel

  As a consumer of the Order Service
  I should be able to place, revise, and cancel an order

  Scenario: Order created, revised, and cancelled
    Given a valid consumer
    Given using a valid credit card
    Given the restaurant is accepting orders
    When I place an order for Chicken Vindaloo at Ajanta
     Then the order should be APPROVED
    Then the order total should be 16.33
    And when I revise the order by adding 2 vegetable samosas
     Then the order total should be 20.97
    And when I cancel the order
    Then the order should be CANCELLED
```

Slayttaki işaretlere göre senaryonun üç parçası vardır: “When I place an order …” ile başlayan kısım siparişi oluşturur (Create Order), “revise the order …” kısmı siparişi revize eder (Revise Order), “cancel the order” kısmı siparişi iptal eder (Cancel Order). Given satırları ön koşulları (geçerli müşteri, geçerli kredi kartı, sipariş kabul eden restoran), When satırları kullanıcı eylemlerini, Then satırları beklenen sonuçları tanımlar.

7. End-to-end testlerin çalıştırılması ve performansı

7.1 Testleri çalıştırma

Slayttaki başlıklar: tüm uygulama ve altyapı hizmetlerinin bir arada çalışması, gerçek kullanıcı deneyimi doğrulaması, altyapı gereksinimi, hizmet ve bağımlılıkların çalıştırılması, araçlar, örnek uygulama (FTGO test modülü), performans sorunları ve çözümleri (ekrandan, 00:22:10).

End-to-end testler bütün uygulama ve altyapı hizmetlerini bir arada çalıştırmayı gerektirir ve uygulamanın gerçek kullanıcı koşullarında nasıl davrandığını doğrular. Testler başlamadan önce birden fazla hizmetin ve veritabanı, API'ler gibi altyapının başlatılması gerekir. Gradle Docker Compose gibi araçlar test sırasında gereken bütün konteynerleri başlatır.

Örnek uygulama olarak FTGO'nun end-to-end test modülü verilir. Burada Gherkin senaryolarıyla yazılmış testler bir araçla çalıştırılır, Gradle Docker Compose eklentisi de testlerden önce gerekli konteynerleri başlatır. Öğrenciye göre basit bir uygulama için bütün konteynerleri başlatıp testleri çalıştırmak 4-5 dakika kadar sürebilir ve bu süre önemli ölçüde uzayabilir.

7.2 Performans sorunları ve çözümleri

Yüzlerce konteyner ve test olduğunda yürütme süreleri çok uzar, yavaş test döngüleri de geliştirme sürecini etkiler. Çözüm olarak daha az ama daha etkili end-to-end test yazılabilir ve daha hızlı geri bildirim için piramidin alt seviyelerindeki testlere odaklanılabilir.

Slayttaki performans iyileştirme stratejileri üç gruptur (ekrandan, 00:24:30):

- Performans sorunları:
  - Yavaşlık: bütün sistem çalıştığı için test süresi uzar, geliştirme ve test döngüleri yavaşlar, ürünün teslim süresi etkilenir.
  - Kırılganlık: birden fazla hizmet ve altyapı olduğu için başarısızlık ihtimali yükselir.
  - Maliyet: yüksek işlem gücü ve kaynak ihtiyacı maliyeti artırır.
- Stratejiler:
  - Kullanıcı yolculuğu testlerine odaklanmak: kritik işlevlere odaklanan senaryolar yazmak, kapsamı artırıp test sayısını azaltmak.
  - Test süresini optimize etmek: testleri paralel çalıştırmak; sık kullanılan hizmetler için ortamı bir kez kurup hep aynı ortamı kullanmak.
  - Test hiyerarşisini dengeli kullanmak: piramidin alt seviyelerindeki testleri artırıp end-to-end testlerin yükünü azaltmak.
  - Testlerin otomasyonu: testlerin manuel müdahale olmadan çalışması.
  - Hataları erken tespit etmek: düzeltme maliyetini düşürür.
- Araçlar ve yöntemler:
  - Paralel çalıştırma: tek makinede 100 testi arka arkaya koşmak yerine testleri birden çok makinede aynı anda koşarak süreyi kısaltmak.
  - Sanallaştırma: sanal makineler ve Docker konteynerleriyle altyapı yükünü azaltmak.
  - Kaynak izleme: performans darboğazlarını bulmak için CPU ve bellek kullanımını izlemek.

7.3 Özelleştirilmiş end-to-end testler

Özelleştirilmiş end-to-end testler, belirli iş süreçlerini ve user journey'leri hedefleyen, kapsamı daraltılmış testlerdir. Genel uçtan uca testlerin aksine yalnız kritik işlevlerin doğru çalıştığını doğrulamaya odaklanırlar. Avantajları:

- Daha hızlı yürütme süresi.
- Daha az hizmete bağımlı oldukları için daha az kırılganlık.
- Kritik alanlara odaklanarak kapsamlı testlere göre daha verimli sonuç.

Hedef, sistem üzerindeki genel test yükünü azaltmaktır. Uygulamanın yalnız belli bir parçasını test eden senaryolar oluşturulur; öğrencinin örneğinde yalnız ödeme işlemini test eden bir senaryo. Testlerin birbirinden bağımsız olması ve her testin kendi altyapı ihtiyacını karşılayabilmesi başarısızlık ihtimalini azaltır. Daha az kapsamlı ama daha sık çalışan testlerle hızlı geri bildirim alınır.

7.4 Sunumun sonucu

End-to-end testler bütün bileşenleri ve entegrasyonları test ederek gerçek kullanıcı deneyimini simüle eder ve sistemin genel işlevselliğini doğrulamada kritik rol oynar. Zorlukları (yavaşlık, kırılganlık, maliyet) test piramidini dengeli kullanarak, user journey testlerine odaklanarak ve özelleştirilmiş testlerle aşılabilir. Her testin amacı ve kapsamı iyi tanımlanmalı, gereksiz testlerden kaçınılmalı ve test sayısı en aza indirilmelidir. Zaman ve kaynak maliyetini azaltmak için end-to-end testler alt seviye testlerle dengeli uygulanmalı, araçlar ve düzgün planlamayla optimize edilmelidir.

8. Üretime hazır servisler ve güvenliğin dört yönü

Üçüncü sunum güvenli servisler geliştirmeyi anlatır. Plan: konuya giriş, güvenli servis geliştirmenin dört yönü, monolitik uygulamalarda güvenlik, mikroservis mimarisinde güvenlik ve sonuç.

8.1 Üç kalite özelliği

Öğrenciye göre bir servisin üretim ortamına dağıtılmaya hazır olması için üç kritik kalite özelliğini karşılaması gerekir:

- Güvenlik: mikroservis mimarisinde kullanıcı kimliğinin servisler arasında güvenli biçimde aktarılması da buna dahildir. Tekil (monolitik) ve mikroservis uygulamalarında benzer mekanizmalar olsa da mikroservislerde ek uygulamalar gerekir.
- Yapılandırılabilirlik (configurability): sistemin farklı ortamlar için gereken bilgileri dışarıdan alabilmesi ve dinamik çalışabilmesi.
- Gözlemlenebilirlik (observability): mikroservislerin dağıtık yapısı yüzünden uygulamanın nasıl çalıştığını anlamak ve sorunları çözmek zordur; izleme ve günlük kaydı gibi yöntemler gerekir.

Sunum bunlardan güvenliğe odaklanır.

8.2 Güvenliğin dört yönü

Slayta göre güvenli yazılım geliştirmek için donanım güvenliği, veri şifreleme, kimlik doğrulama ve yazılım güncellemeleri gibi çeşitli önlemler alınmalıdır; bu önlemler tekil ya da mikroservis mimarisinde benzer biçimde ele alınır (ekrandan, 00:30:00). Dört yön:

- Kimlik doğrulama (authentication): uygulamaya erişmeye çalışan insanların ya da uygulamaların kimliğini doğrulama süreci. Genellikle kullanıcı adı, şifre ya da API anahtarı gibi kimlik bilgileriyle yapılır.
- Yetkilendirme (authorization): bir kullanıcının ya da uygulamanın belirli veriler üzerinde işlem yapma yetkisine sahip olup olmadığını doğrulayan süreç. Genellikle rol tabanlı güvenlik ve erişim kontrol listeleri (ACL) ile kullanıcıların ya da rollerin yetkileri belirlenir.
- Denetleme (auditing): güvenlik sorunlarını tespit etmek, müşteri desteğini kolaylaştırmak ve uyumluluğu sağlamak için yapılan işlem izleme süreci.
- Güvenli süreçler arası iletişim: servisler arası iletişim ideal olarak Transport Layer Security (TLS) ile güvence altına alınmalı ve kimlik doğrulamayla desteklenmelidir.

  Ders sonrası not: Öğrenci TLS'i anlatırken "aslında HTTP'yi kullandığımız" bir süreç dedi. Doğrusu: TLS, HTTP'nin altında çalışan bir şifreleme katmanıdır; HTTP'nin TLS üzerinden kullanılması HTTPS'tir. Düz HTTP tek başına iletişimi şifrelemez.

9. Monolitik uygulamada güvenlik ve mikroservislere uygulanamaması

9.1 Genel görünüm

Monolitik bir uygulamada (FTGO) istemci, yani tarayıcı ya da mobil uygulama, kullanıcı adı ve şifreyle giriş yapar (ekrandan, 00:30:40). Uygulama kimliği doğrular ve başarılı girişte istemciye oturum belirteci (session token) içeren bir çerez döner. Bu oturumda kullanıcının bilgileri, rolleri ve yetkileri bulunur. İstemci sonraki isteklerine bu çerezi ekler; böylece uygulama işlemi yapan kişinin kim olduğunu bilir.

9.2 Adım adım akış

Slayttaki ayrıntılı şema uygulamanın içini gösterir (ekrandan, 00:31:00):

```
1) Login-based client --POST /login (userId=...&password=...)--> Login handler
2) Login handler: kullanıcı bilgisini User database'den okur,
   Session'ı başlatır (UserId, roles: [CONSUMER])
3) Login handler --HTTP/1.1 200 OK, Set-cookie: JSESSIONID=...--> client
4) client --GET /orders/order-xyz, Cookie: JSESSIONID=...--> SessionBasedSecurityInterceptor
5) SessionBasedSecurityInterceptor: Session'ı okur, Security context'i kurar
6) OrderDetailsRequestHandler: Security context'i okur ve isteği işler
```

Öğrencinin anlatımıyla: istemci kullanıcı bilgilerini uygulamanın içindeki login handler'a gönderir. Login handler kimlik bilgilerini veritabanından doğrular; başarılı olursa oturuma kullanıcı kimliğini, rolünü ve yetkilerini yazar. Üçüncü adımda oturum bilgisi istemciye geri gönderilir. Dördüncü adımda istemci bu oturum belirtecini sonraki isteklerine ekler. Ardından bir güvenlik bağlamı (security context) kurulur ve altıncı adımda kullanıcının talep ettiği işlemin yapılıp yapılmayacağına bu bağlama göre karar verilir.

9.3 Mikroservislere neden uygulanamaz?

Bu yaklaşım dağıtık yapıya uygun olmadığı için mikroservis mimarisine doğrudan uygulanamaz. İki temel sebep vardır:

- Bellek içi güvenlik bağlamı (in-memory security context): monolitte güvenlik bağlamı ve kullanıcı kimliği işlem sırasında bellekte taşınır. Mikroservislerde her hizmet bağımsız çalışır ve paylaşılan bir bellek yoktur.
- Merkezi oturum: monolitte kullanıcı bilgileri bellekteki merkezi oturumda tutulur. Mikroservislerde hizmetlerin böyle bir merkezi oturuma bağımlı olması gevşek bağlılık (loose coupling), yani sistem bileşenlerinin birbirine en az düzeyde bağlı olması ilkesini ihlal eder.

10. API Gateway'de kimlik doğrulama ve yetkilendirme

10.1 Kimlik doğrulama nerede yapılmalı?

İlk seçenek, her servisin kimlik doğrulamasını kendisinin yapmasıdır. Öğrenciye göre bu tavsiye edilmez:

- Kimliği doğrulanmamış isteklerin iç ağa girmesine izin verdiği için büyük bir güvenlik açığı içerir.
- Farklı istemcilerin farklı kimlik doğrulama yöntemleri olabilir; bunları her serviste ele almak karmaşıklığa ve performans düşüşüne yol açar.

Bu yüzden kimlik doğrulama API Gateway'de yapılır. Böylece:

- Güvenlik tek bir noktada toplanır ve güvenlik açığı riski azalır.
- Farklı kimlik doğrulama mekanizmalarıyla servisler değil gateway ilgilenir; karmaşıklık azalır, servislerin performansı artar.

Şemadaki akışta API istemcisi kimlik bilgilerini gateway'e gönderir. Gateway doğrularsa sonraki isteklere kimlik bilgilerini içeren bir güvenlik belirteci (security token) ekler ve istek ulaşılmak istenen servise iletilir. Oturum tabanlı istemcilerde mantık aynıdır: kimlik bilgileri gateway'e gönderilir, doğrulanırsa istemciye güvenlik belirteci döner, istemci bu belirteci sonraki isteklerine ekler, gateway de isteği ilgili servise iletir.

10.2 Yetkilendirme

İstemcinin kimliğini doğrulamak önemlidir ama tek başına yeterli değildir. Uygulama ayrıca istemcinin talep ettiği işlemi yapmaya yetkili olduğunu doğrulayan bir yetkilendirme mekanizması uygulamalıdır.

Yetkilendirmeyi de gateway'de uygulamanın en büyük avantajı, güvenliği merkezileştirerek riski önemli ölçüde azaltmasıdır. Öğrencinin saydığı dezavantajlar:

- Gateway'in servislerle sıkı bağlı olma riski artar; servisler ve gateway'in eşzamanlı güncellenmesi gerekir.
- Gateway yalnız URL'ye dayalı, rol tabanlı erişim kontrolü sağlayabilir; karmaşık erişim kontrolünü desteklemez.

Öğrencinin yemek uygulaması (FTGO) örneğinde, bir kullanıcının siparişlere erişip erişemeyeceği gateway'de kontrol edilebilir. Ancak "her kullanıcı yalnız kendi siparişlerine ulaşsın" kuralı gateway'de sağlanamaz. Bu tür kurallar servislerin içinde bulunan erişim kontrol listeleriyle (ACL) uygulanır; böylece yetkilendirme servislerde de yapılır ve karmaşık işlemler desteklenir.

11. Belirteç türleri: opak belirteçler ve JWT

Gateway'in kullanıcı bilgisini servislere iletmek için hangi tür belirteci kullanacağına karar vermek gerekir. İki seçenek vardır (ekrandan, 00:37:05).

11.1 Opak belirteçler

Opak (opaque) belirteç dışarıdan bakıldığında hiçbir bilgi içermez. Kullanıcı bilgisine erişmek için ayrı bir güvenlik servisine eşzamanlı RPC (Remote Procedure Call) çağrısı yapılır. Her istek için ek bir doğrulama çağrısı gerektiğinden performans düşer; ayrıca güvenlik servisinde oluşan bir hata bütün sistemi etkileyebilir. Bu yüzden opak belirteçler tavsiye edilmez.

11.2 Şeffaf belirteçler ve JWT

Şeffaf (transparent) belirteç kullanıcı hakkında bilgi içerir; servis belirteci kendisi doğrulayabilir. En popüler şeffaf belirteç formatı JSON Web Token'dır (JWT). Öğrenciye göre JWT iki kısımdan oluşur:

- Payload: kullanıcının kimliği ve rolleri gibi bilgileri içerir.
- İmza (signature): belirtecin bütünlüğünü sağlar.

JWT ile RPC çağrısı ortadan kalkar ve performans artar. Dezavantajı, verilmiş bir belirtecin geri alınamamasıdır (iptal edilememesi); bu güvenlik açığı riskini artırabilir. Çözüm, JWT'leri kısa ömürlü oluşturmaktır. Kısa ömür güvenliği artırır ama bu kez oturumu açık tutmak için belirtecin sürekli yenilenmesi gerekir.

> **Ders sonrası not:** Öğrenci imzanın belirtecin "bütünlüğünü ve gizliliğini" sağladığını söyledi. Doğrusu: imza yalnız bütünlüğü ve belirteci kimin ürettiğini doğrular; gizlilik sağlamaz. İmzalı bir JWT'nin payload'ı yalnız Base64URL ile kodlanmıştır ve herkes okuyabilir. Gizlilik gerekiyorsa büyük olasılıkla şifreli JWT (JWE) ya da TLS kullanılır.
>
> **Ek bilgi (derste anlatılmadı):** İmzalı bir JWT aslında noktalarla ayrılmış üç parçadan oluşur: `header.payload.signature`. Header imza algoritmasını (örneğin HS256 ya da RS256), payload kullanıcı bilgilerini ve son kullanma zamanı (`exp`) gibi alanları, signature ise ilk iki parçanın imzasını taşır.

12. Mikroservis mimarisinde OAuth 2.0

12.1 Temel kavramlar

OAuth (Open Authorization) 2.0'ın asıl amacı genel bulut hizmetlerine erişimi yetkilendirmektir; öğrencinin örneğinde Google'da ve bulut hizmetlerinde kullanılan güvenlik standardıdır. Uygulamalarda kimlik doğrulama ve yetkilendirme için de kullanılabilir. Temel kavramlar (ekrandan, 00:39:15):

- Yetkilendirme sunucusu (Authorization Server): kullanıcıların kimliğini doğrulamak ve bir erişim belirteci (access token) ile yenileme belirteci (refresh token) almak için bir API sağlar.
- Erişim belirteci (access token): bir kaynak sunucusuna erişim izni veren belirteçtir. Formatı uygulamaya bağlıdır, genellikle JWT kullanılır. Öğrenciye göre bu, önceki bölümlerdeki güvenlik/oturum belirteci gibi kimlik bilgisi taşır.

- Yenileme belirteci (refresh token): uzun ömürlü ama iptal edilebilir bir belirteçtir. İstemci yeni bir erişim belirteci almak için bunu kullanır.
- Kaynak sunucusu (resource server): mikroservis mimarisinde hizmetler kaynak sunucusu işlevi görür; yani ulaşılmak istenen servislerdir.
- İstemci (client): mikroservis mimarisinde API Gateway, OAuth 2.0 istemcisi olarak işlev görür.

12.2 API istemcisinde kimlik doğrulama akışı

Slayttaki akış dört adımdır (ekrandan, 00:40:00):

```
1) API client --GET /orders/1, Authorization: Basic ...--> API gateway
2) API gateway --POST /oauth/token (userid=...&password=...)--> Spring OAuth2 Authentication S
   (Password grant request; sunucu User database'i kullanır)
3) Authentication Server --HTTP/1.1 200 OK {"access_token": "AccessToken", ...}--> API gateway
4) API gateway --GET /orders/1, Authorization: Bearer AccessToken--> Order Service
   (belirteç kullanıcı kimliğini ve rollerini içerir)
```

İstemci temel kimlik doğrulamayla kimlik bilgilerini içeren bir istek gönderir. Gateway OAuth 2.0 yetkilendirme sunucusuna bir Password Grant isteği yapar. Sunucu istemcinin kimlik bilgilerini kendi veritabanından doğrular ve bir erişim belirteci döndürür. Gateway yaptığı hizmet isteklerine bu erişim belirtecini ekler.

12.3 Oturum tabanlı istemcilerde akış

Durum benzerdir. Kullanıcının kimlik bilgileri gateway'in login handler kısmına iletilir; oradan OAuth 2.0 sunucusuna gönderilir. Doğrulanırsa belirteç gateway'e, oradan istemciye döner. İstemci bu belirteci sonraki isteklerine ekler; gateway de istekleri ulaşılmak istenen hizmete iletir.

12.4 Sunumun sonucu

Bir hizmetin işlevsel gereksinimlerini karşılaması önemlidir, ama aynı zamanda güvenli, yapılandırılabilir ve gözlemlenebilir olmalıdır. Mikroservis güvenliğinin birçok yönü tekil mimarideki gibidir, ancak ek önlemler gerekir. Yaygın yaklaşımda istemciler kimlik doğrulamasını API Gateway ile yapar; gateway her servis isteğinde JWT gibi şeffaf bir belirteç kullanır. OAuth 2.0 da mikroservis mimarisi için güçlü bir güvenlik temeli sağlayan standarttır.

13. Microservice Chassis deseni

Dördüncü sunumun konusu "Developing Services Using the Microservice Chassis Pattern"dir.

13.1 Problem: kesişen kaygılar

Mikroservis uygulaması, sunduğu işlevselliğe göre bölünmüş bir dizi küçük, bağımsız servisten oluşur. Bununla birlikte her mikroservisin karşılaması gereken ve bütün servislerde tekrarlanan bir dizi kesişen kaygı (cross-cutting concerns) vardır: hata yönetimi, loglama, performans takibi, izleme (monitoring), güvenlik gibi. Servis sayısı arttıkça bu ortak işlerin her serviste yeniden yazılması kod tekrarına, fazlalığa ve kaynak israfına yol açar.

13.2 Çözüm: chassis

Microservice Chassis deseni bunu önlemek için ortaya atılmıştır. Bu yaygın kaygıları yöneten, bütün bu özellikleri sağlayan ve üzerine iş işlevselliği eklenebilecek bir iskelet uygulama (framework) oluşturmak fikridir. Chassis, servislerin hızlı ve yalın biçimde geliştirilmesini sağlayan bir çatı olarak tanımlanır. Spring Boot ve Spring Cloud, Microservice Chassis'in klasik örnekleridir. Öğrenciye göre mutlaka hazır bir framework gerekmez; bir kütüphane oluşturularak da framework simüle edilebilir ve desen uygulanabilir.

Öğrencinin benzetmesiyle, bir aracın şasisi sabit tutulur ve üzerine tekerlek, farlar gibi parçalar takılır. Mikroservis bağlamında chassis, temel bir framework ya da farklı servislerde

yeniden kullanılabilen başka bir servis olabilir. Kesişen kaygıların temel çerçevesinin yeniden kullanılmasını sağlar; her servis yalnız kendi iş işlevselliğine odaklanır, ortak işler kesişim noktasında çözülür.

13.3 Diyagram

Slayttaki diyagram chassis modelini kullanan tipik bir mikroservis geliştirmeyi gösterir (ekrandan, 00:44:58):

```
Shared Libraries:  [Messaging]  [DB Connectivity]
                         ^              ^
                         |              |  (yalnız gereken servis)
Service A ---------+----+--------------+
Service A ---------+--> Chassis Skeleton
Service A ---------+      Service Runtime --> Logging
                          Service Runtime --> Monitoring
                          Service Runtime --> Health Check
                          Service Runtime --> Service Discovery
```

Bütün servisler iskelet chassis'i kullanır. Slayta göre yalnız mesajlaşma desteği ve veritabanı bağlantısı gerektiren servis, bu amaçla geliştirilmiş bağımsız paylaşımlı kütüphaneleri (Messaging, DB Connectivity) ayrıca kullanır.

13.4 Avantajlar ve dezavantaj

Avantajlar:

- Yazılması gereken kod miktarını önemli ölçüde azaltır.
- Yapısal sorunları ortadan kaldırarak iş mantığının geliştirilmesine odaklanmayı sağlar.
- Diğer mikroservislerde yeniden kullanılabilen kararlı bir kod tabanı sağlar.
- Yeni özellikler iskelete bağımsız olarak eklenebilir; yalnız bu özellikleri gerektiren servislerin güncellenmesi gerekir. Diğer servisler ya hiç etkilenmez ya da en az düzeyde etkilenir.

Dezavantaj: servisleri geliştirmek için kullanılan her dil ya da platform birleşimi için ayrı bir chassis geliştirmek gerekir.

13.5 Öğrencinin loglama örneği

Her istek geldiğinde, isteğin geldiği anın bir log dosyasına yazılması istenir. Her istek için yeniden bir logger oluşturup "bu logu şu dosyaya yaz" demek hem kod tekrarıdır hem de kodu karmaşıklaştırır (öğrencinin ifadesiyle "spagetti" oluşturur). Bunun yerine log yapısı sabit bir taban sınıfta kurulur ve istekler bu yapıdan kalıtımla türetilir. Gelen her istek önce taban kısımda yakalanır, log basılır; ardından istek neyse (sağlık kontrolü, loglama vb.) onun iş mantığı çalıştırılır ve cevap döndürülür.

14. Service Mesh

14.1 Tanım

Öğrenci chassis'ten Service Mesh'e bir köprü kurar. Service Mesh, servislere giren ve çıkan bütün ağ trafiğini yöneten bir ağ katmanıdır. Devre kesiciler (circuit breaker), dağıtık izleme (distributed tracing), servis keşfi, yük dengeleme ve kural tabanlı trafik yönlendirme gibi işleri bu katman uygular.

Chassis'in dezavantajı her dil için ayrı geliştirme gerektirmesiydi. Service Mesh bunu önleyen bir alternatiftir: bu işlerin bir kısmı servisin dışında, belirli bir platforma bağlı olmadan uygulanır. Mesh, bir servis ile diğer servisler ve harici uygulamalar arasındaki iletişime aracılık eden bir ağ altyapısıdır; servise giren ve çıkan bütün trafik mesh üzerinden geçer. Öğrencinin anlatımıyla yük dengeleme ve güvenli iletişim gibi işler önce alt katmanda yapılır, ardından istek ilgili mikroservise düşer.

Ek bilgi (derste anlatılmadı): Yaygın Service Mesh gerçekleştirmeleri arasında Istio ve Linkerd bulunur. Bunlar genellikle her servisin yanına bir vekil (sidecar proxy, örneğin Envoy) yerleştirir ve servisin bütün trafiği bu vekil üzerinden geçer.

14.2 Dağıtımı sürümden ayırma

Slayttaki özet (ekrandan, 00:47:10):

- Service Mesh de Microservice Chassis gibi geliştiriciyi kesişen kaygılarla uğraşmaktan kurtarır.
- Mesh'in trafiği yönlendirme yeteneği, dağıtımı (deployment) sürümden (release) ayırmayı mümkün kılar.
- Bir hizmetin yeni sürümünü üretime dağıtabilir, ama onu yalnız dahili test kullanıcıları gibi belirli kullanıcılara açabilirsiniz.

14.3 Mimari ve SSL örneği

Öğrenci son olarak mikroservis ile mesh birleştirilmiş altyapıyı karşılaştırır. Yalnız mikroservis yapısında istek doğrudan bir servisten diğerine geçer. Mesh ile birleştirildiğinde istekler önce bir alt katmanda, vekillerde (proxy) toplanır; metrikler ve yapılandırmaya göre işlem yapılır ve isteğe göre yanıt dönülür.

Mesh için verilen örnek SSL'dir. Gelen istek doğrudan alt katmanda yakalanır: güvenli bir altyapıdan mı gelmiş, sertifikalı bir istek mi, şifresi doğrulanabiliyor mu diye bakılır. İstek güvenliyse alt katman onu kabul eder; güvenli olduğu netleştikten sonra isteğin işi neyse ilgili servis onu işler ve cevap döner. Böylece güvenlik denetimi servis kodunda değil, altındaki katmanda yapılmış olur.

Dersin Toparlaması

Bu derste dört öğrenci sunumuyla mikroservis kitabının sorgu tarafı tasarımı, test ve üretime hazırlık bölümleri işlendi. CQRS view modülünde Query API ve event handler'lar view veritabanına bir veri erişim modülü üzerinden erişir. Veritabanı sorgu türüne göre seçilir; CQRS'in basit işlemleri için NoSQL genelde iyi bir seçimdir. Veri erişim modülü eşzamanlı güncellemeleri kilitlemeyle, tekrarlanan event'leri eşsiz event kimlikleriyle ele alır; replikasyon gecikmesine karşı istemciye event kimliği içeren bir token verilir. Yeni bir view arşivlenmiş event'lerden ya da snapshot tabanlı artımlı algoritmayla oluşturulur.

End-to-end testler test piramidinin tepesindedir; değerlidir ama yavaş, kırılgan ve pahalıdır. Bu yüzden az sayıda, user journey odaklı ve Gherkin gibi iş dilinde yazılmaları, paralel çalıştırılmaları ve yükün alt seviye testlere verilmesi önerildi. Güvenlik sunumunda monolitteki oturum ve bellek içi güvenlik bağlamının mikroservislere uygulanamadığı, kimlik doğrulamanın API Gateway'de yapılıp kullanıcı kimliğinin JWT gibi şeffaf belirteçlerle servislere taşındığı ve OAuth 2.0'da gateway'in istemci, servislerin kaynak sunucusu olduğu anlatıldı. Son sunumda kesişen kaygılar için dile bağlı bir çatı olan Microservice Chassis ile bu işleri dil bağımsız olarak ağ katmanında üstlenen Service Mesh karşılaştırıldı. Planlanan iki sunum sonraki derse kaldı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.